Iterating on `parallel_scheduler`

Document #:	P3804R2
Date:	2026-03-26
Project:	Programming Language C++
Audience:	LWG

Reply-to:	Lucian Radu Teodorescu (Garmin) <lucteo@lucteo.ro> Ruslan Arutyunyan (Intel) <ruslan.arutyunyan@intel.com>

Contents

§ Abstract

`parallel_scheduler` [P2079R10] was a long time in the making and was it adopted in Sofia 2025; still more design concerns were raised after that. This paper proposes to iterate on some of these aspects, aiming to achieve the best possible outcome from `parallel_scheduler`.

§ 1 Changes

§ 1.1 R2

— incorporate the feedback from LWG, Croydon 2026

§ 1.2 R1

— incorporate the feedback from SG1 + LEWG, Kona 2025

— rebase wording on top of the decisions taken for [P3826R1]

§ 2 Introduction

This paper tries to address the following concerns:

— Minor design tweaks:

1. `receiver_proxy::try_query` could possibly be `const`-qualified.
2. `receiver_proxy` does not need a virtual destructor as the object is never destroyed polymorphically.
3. `receiver_proxy::try_query` requires `inplace_stop_token` and doesn't accept an arbitrary stop token.
4. The list of properties supported by `receiver_proxy::try_query` is implementation-defined.

— Design concerns with larger impact:

5. `receiver_proxy::try_query` currently requires the list of supported queries to be defined.

— Naming:

6. `system_context_replaceability` is not a good name to use for the namespace in which the replaceability APIs lie.

— Wording:

7. The wording around customizations of `bulk_unchunked`/`bulk_chunked` for `parallel_scheduler` isn't precise enough.

§ 3 Discussions

§ 3.1 `receiver_proxy::try_query` could possibly be `const`-qualified

Currently, the specification defines `try_query` as:

```
template<class P, class-type Query>
optional<P> try_query(Query q) noexcept;
```

This is not marked as `const`, but there is no good reason why we couldn't mark it as such. This paper proposes a change to mark this function as `const`.

§ 3.2 `receiver_proxy` does not need a virtual destructor

The code that destroys instances of `receiver_proxy` knows the actual type of the object, so objects of type `receiver_proxy` don't need to be destroyed polymorphically. This paper proposes to remove the virtual destructor.

§ 3.3 `receiver_proxy::try_query` requires `inplace_stop_token` and doesn't accept an arbitrary stop token

The specification of `parallel_scheduler::try_query` [P2079R10] is too restrictive with respect to querying stop tokens. The wording states (33.16.2 [exec.sysctxrepl.query]):

```
template <class P, class-type Query>
optional<P> try_query(Query q) noexcept;
```

5 *Mandates*: `P` is a cv-unqualified non-array object type.

6 *Returns*: Let `env` be the environment of the receiver represented by `*this`. If:

(6.1) — `Query` is not a member of an implementation-defined set of supported queries; or

(6.2) — `P` is not a member of an implementation-defined set of supported result types for `Query`; or

(6.3) — the expression `q(env)` is not well-formed or does not have type *cv* `P`,

then returns `nullopt`. Otherwise, returns `q(env)`.

7 *Remarks*: `get_stop_token_t` is in the implementation-defined set of supported queries, and `inplace_stop_token` is a member of the implementation-defined set of supported result types for `get_stop_token_t`.

This implies that, if `q(get_stop_token_t)` returns `std::stop_token`, then `try_query(get_stop_token_t)` returns `nullopt`. More generally, if `q(get_stop_token_t)` returns a type that models `stoppable_token`, other than `inplace_stop_token`, then `try_query(get_stop_token_t)` returns `nullopt`. There is no portable way for the backend to check the stop token of the receiver.

This paper proposes an extension of the above schema to allow the possibility of using stop tokens other than `inplace_stop_token`. If `q(env)` has the type *cv* `T`, and there is an implementation-defined mapping from objects of type `T` to objects of type `P`, then `try_query<P>(q)` is allowed to return non-null objects.

We recommend that implementations support such mappings between any stop token and `inplace_stop_token`. This would essentially make the frontend register a stop callback to the token from the environment and transform the stop request into a stop request to a temporary `inplace_stop_token`.

Please note that this mechanism is useful for other property types, not just stop tokens.

We also considered an alternative design in which we add a `request_stop` member function to the backend. The frontend could register a stop callback to the receiver's stop token and directly call this function in the backend. Then, the backend could run whatever action is necessary to cancel the allocation of a thread.

The main advantage of this alternative is the reduction of the number of stop callbacks that need to be registered, thus reducing the size of the operation state.

In our proposed solution, if the receiver's stop token is not an `inplace_stop_token` then we have to adapt it to such an object. This adaptation requires the use of a stop callback. Then, in the backend, we need another stop callback to be able to transform the received `inplace_stop_token` into a call to the underlying thread pool (Windows Thread Pool, libdispatch, etc.) to cancel outstanding work. This means that, in the worst case, we would use two stop callbacks (in the best case, we are using only one).

In this alternative, we would always use a stop callback, thus we might be better off. However, this alternative has a number of disadvantages:

— Increases API surface (we need another backend class with a method dedicated to requesting stop).

— It is a property that can be queried from the receiver, yet we introduce a special API that leads to non-uniform behavior with querying other properties.

— If we can't query stop tokens through the `try_query` interface, for C++26 we won't have any properties that would use `try_query`.

— May lead to larger storage needs in the operation state.

Currently, the API for `parallel_scheduler_backend` looks like:

```
struct parallel_scheduler_backend {
  virtual ~parallel_scheduler_backend() = default;

   virtual void schedule(receiver_proxy&, std::span<std::byte>) noexcept =
        0;
    virtual  void  schedule_bulk_chunked(size_t,  bulk_item_receiver_proxy&,
        std::span<std::byte>) noexcept = 0;
   virtual void schedule_bulk_unchunked(size_t, bulk_item_receiver_proxy&,
        std::span<std::byte>) noexcept = 0;
};
```

If we move towards this new alternative, we would need the following change:

```
struct backend_operation {
  virtual void request_stop() noexcept = 0;
};

struct parallel_scheduler_backend {
  virtual ~parallel_scheduler_backend() = default;

          virtual     voidbackend_operation*     schedule(receiver_proxy&,
        std::span<std::byte>) noexcept = 0;
        virtual    voidbackend_operation*    schedule_bulk_chunked(size_t,
        bulk_item_receiver_proxy&, std::span<std::byte>) noexcept = 0;
       virtual    voidbackend_operation*    schedule_bulk_unchunked(size_t,
        bulk_item_receiver_proxy&, std::span<std::byte>) noexcept = 0;
};
```

If we just look at the space consumption, we can conclude the following:

— We need a virtual table, so that is a pointer occupied in the operation state;

— The frontend needs to store a pointer to the returned `backend_operation` object, so that is another pointer.

These two pointers typically occupy less than a stop callback, so this alternative might still occupy less space for the most general case. But, for the case in which the receiver has an `inplace_stop_token`, this actually occupies more space. Thus, the space advantage of this alternative is not a net win.

Considering API surface, uniformity, continued usefulness of `try_query`, and storage trade-offs, we conclude that the proposed extension is preferable to the `request_stop` alternative.

§ 3.4 The list of properties supported by `receiver_proxy::try_query` is implementation-defined

When [P2079R10] defines the possibilities for the backend to query the properties of the final receiver, it specifies that the list of properties (both property types and query types) are implementation-defined. Instead, the proposal could have defined a fixed list of properties to be supported when replacing the backend.

That seems to be a limiting factor as we would want to support two use-cases:

1. Implementers might want to add additional properties to be queried.
2. Implementers might not want to implement all the properties.

A good example for the first case is a vendor adding support for thread priorities. With the existing proposal, this can be done in a compliant manner.

The second case involves implementers opting out of certain properties. Let us assume that we standardize a mechanism to query thread priorities from the receiver. But this may not make much difference on certain platforms, thus the implementers should be able to opt out of supporting this query. Supporting a query typically has a small cost associated with it, so it makes sense to allow opting out of this cost if it doesn't make sense for the targeted platform.

Even with the stop-token property that is specified by [P2079R10], it may not always make sense to support cancellation on the backend. The frontend can implement cancellation without passing this information to the backend.

If we keep the list of properties (and query types) to be implementation-defined and fixed, we can always relax this later. Relaxing this in a later standard will imply that we would need to support new properties; this would be similar to adding new properties to be supported.

Thus, the authors believe that the current option of making the list of properties (and query types) implementation-defined is the right choice for the users and no changes are needed here.

§ 3.5 `receiver_proxy::try_query` currently requires the list of supported queries to be defined

One workflow that some C++ users have is to have separate compilations of their libraries. Two libraries, A and B, can be compiled separately, sometimes with different versions of the compiler, sometimes even with different compilers (provided that the ABI matches). The `parallel_scheduler` feature cannot break this flow.

For practical reasons, we can consider the backend of `parallel_scheduler` as yet another library C that can be compiled separately from the rest of the program. The backend may be compiled with different flags and with a different compiler than the rest of the libraries (again, if the ABI matches). That is, there is a complete separation between the frontend (library implementing `parallel_scheduler`) and the backend (user-provided implementation of `parallel_scheduler_backend`). Things are more complicated as the user code (the environment of the receiver connected to a `parallel_scheduler` sender) is outside of the control of the standard library.

Thus, we have 3 components that we need to align:

— user code (receiver's environment),

— frontend (`parallel_scheduler` implementation in the standard library),

— backend (user-supplied functionality for launching execution agents).

One attempt at trying to bridge these three things might look like:

```
// backend code
struct receiver_proxy {
  template<typename R, typename Q>
  std::optional<R> try_query(Q) {
    alignas(R) std::byte storage[sizeof(R)];
    if (try_query_impl(typeid(Q), typeid(R), &storage)) {
       struct dtor {
          R& result;
          ~dtor() { result.~R(); }
       };
       dtor d{*std::launder(reinterpret_cast<R*>(&storage))};
       return std::move(d.result);
    }
    return std::nullopt;
  }
private:
    virtual bool try_query_impl(std::type_index query_id, std::type_index
        result_id, void* result_addr) = 0;
};

// frontend code
template<typename Receiver>
struct receiver_proxy_impl : receiver_proxy {
   using env_t = std::env_of_t<Receiver>;
   using queries_t = queries_of_t<env_t>;
   template<typename Q>
      using  query_result_t  =  decltype(auto(std::declval<const  env_t&>
        ().query(Q{})));

   Receiver rcvr;

   struct vtable_entry {
     std::type_index query_id;
     std::type_index result_id;
     void (*getter)(receiver_proxy*, void* address);
   };

   template<typename... Queries>
   static constexpr auto make_query_vtable(type_list<Queries...>) {
      return std::array<vtable_entry, sizeof...(Queries)> vtable{
        {typeid(Queries),
         typeid(query_result_t<Queries>),
         [](receiver_proxy* proxy, void* address) {
                                ::new   (address)   query_result_t<Queries>
        (get_env(static_cast<receiver_proxy_impl*>(proxy)-
        >receiver).query(Queries{}));
         }}...};
   }

   bool try_query_impl(std::type_index query_id,
```

```
                        std::type_index result_id,
                        void* address) {
    static constexpr auto vtable = make_query_vtable(queries_t{});
    for (auto& entry : vtable) {
      if (entry.query_id == query_id && entry.result_id == result_id) {
        entry.getter(this, address);
        return true;
      }
    }
    return false;
  }

  // ... etc. for set_value, and other methods
};
```

The key for this implementation is the `make_query_vtable` function in the frontend that makes a vtable-like structure containing ways to access the properties of the receiver's environment. But this requires that the frontend has access, at compile-time, to the list of properties that the receiver supports. In our example, this is represented by `queries_of_t<env_t>`, which is not detailed here.

The problem is that we don't yet have a good way to implement this query. To fully support all the properties that the receiver has, the frontend needs to be able to build them into a type list. We don't have support for this.

Without such a facility, the frontend and the backend can only query a set of properties that it knows. This aligns with the direction proposed by [P2079R10].

If we were to find such a solution in the future, the frontend could support new properties, which could be picked up by the backend. As this is a relaxation of constraints, future standards can do it without breaking changes.

The authors don't see any changes that would benefit users in a tangible way in this area.

§ 3.6 `system_context_replaceability` is not a good name to use for the namespace in which the replaceability APIs lie

Previously, `parallel_scheduler` was called `system_scheduler`, and was part of `system_context`. In that sense, the name `system_context_replaceability` made sense; it was the namespace in which we put things related to replacing the default implementation around `system_context`. But now, we ended up with a different name, so the question is whether `system_context_replaceability` still makes sense.

There are people whose answer would be no. In that case, we might rename the namespace to something like `parallel_scheduler_replaceability`.

But, there are also arguments for keeping the current name. We also envision having different types of schedulers. We were previously talking about a `main_scheduler` to be

used on systems that have only one thread, or in which the main thread needs special treatment. We also had brief discussions about the need for an `io_scheduler` (or `elastic_parallel_scheduler`) and a `priority_scheduler` (to create threads with different priorities).

There is a high chance that we would add new system-wide schedulers in the future. But, if that is the case, and we want their backends to be replaceable, should we create completely different namespaces for them? Or should we reuse the abstractions that we already have?

Probably, a good answer would be that we want to reuse the same namespace. In this context, the name `system_context_replaceability` makes sense. Especially since the word `context` appears in the name, not the word `scheduler`.

The following table shows a few options:

Alternative names	Notes
`system_context_replaceability`	Status quo. Allows replacing other backends in the future
`parallel_scheduler_replaceability`	Better matches the new name
`scheduler_replaceability`	Variation
`scheduler_backend_replaceability`	Variation
`parallel_scheduler_backend_replaceability`	Variation
`replacement`	Simpler name; can be extended in the future
`replacement_functions`	Variation
`psr`	Abbreviated name from `parallel_scheduler_replaceability`
`scr`	Abbreviated name from `system_context_replaceability`
N/A	Just remove the namespace

In R0, the authors favored the `replacement` option, but would like this to be discussed in LEWG.

After polling in LEWG, we did not get a consensus for change. We keep the namespace as is.

§ 3.7 The wording around the customizations of `bulk_unchunked/bulk_chunked` for `parallel_scheduler` isn't precise enough

The wording in [P2079R10] mandates that we want customizations for `bulk_unchunked/bulk_chunked`, but it did not describe the details. Actually, this discussion was completely missing from the design section, too. The proof-of-concept

implementation used the early customization mechanism, and part of the authors assumed that was always the case, but the paper doesn't mention anything about it. This needs to be fixed.

Also, the customization part does not treat the execution policy at all. Again, this needs to be fixed.

P3826R0 proposes to defer adding customizations to a later standard, and proposes a solution for allowing `parallel_scheduler` to customize the behavior of `bulk_chunked` and `bulk_unchunked`. We aim to build on top of P3826R0.

§ 4 Proposed wording

[Editor's note: In section *Parallel scheduler* [exec.par.scheduler], apply the following changes:]

7 ~~*A bulk chunked proxy for `rcvr` with callable `f` and arguments `args`* is a proxy `r` for `rcvr` with base `system_context_replaceability::bulk_item_receiver_proxy` such that `r.execute(i, j)` for indices `i` and `j` has effects equivalent to `f(i, j, args...)`.~~

8 ~~*A bulk unchunked proxy for `rcvr` with callable `f` and arguments `args`* is a proxy `r` for `rcvr` with base `system_context_replaceability::bulk_item_receiver_proxy` such that `r.execute(i, i+1)` for index `i` has effects equivalent to `f(i, args...)`.~~

9 Let `b` be *`BACKEND-OF`*`(sch)`, let `sndr` be the object returned by `schedule(sch)`, and let `rcvr` be a receiver. If `rcvr` is connected to `sndr` and the resulting operation state is started, then:

- If `sndr` completes successfully, then `b.schedule(r, s)` is called, where:
  - `r` is a proxy for `rcvr` with base `system_context_replaceability::receiver_proxy`; and
  - `s` is a preallocated backend storage for `r`.
- All other completion operations are forwarded unchanged.

? The expression `get_domain(sch)` returns an expression of exposition-only type *`parallel-scheduler-domain`*, that is equivalent with:

```
struct parallel-scheduler-domain {
  template<sender-for<bulk_chunked_t> Sndr, queryable Env>
    static constexpr decltype(auto) transform_sender(set_value_t, Sndr&&
        sndr, const Env& env) const noexcept {
    return see below;
  }
  template<sender-for<bulk_unchunked_t> Sndr, queryable Env>
    static constexpr decltype(auto) transform_sender(set_value_t, Sndr&&
        sndr, const Env& env) const noexcept {
    return see below;
```

```
  }
};
```

? For argument sndr of the above transform_sender, let child, pol, shape and f be defined by the following declarations:

```
auto& [_, data, child] = sndr;
auto& [pol, shape, f] = data;
```

? Let p be:

— true, if the type of the expression pol is *cv* parallel_policy or *cv* parallel_unsequenced_policy;

— implementation-defined, if pol is an implementation-defined execution policy;

— false, otherwise.

10 ~~parallel_scheduler provides a customized implementation of bulk_chunked algorithm (33.9.12.11 [exec.bulk]). If a receiver rcvr is connected to the sender returned by bulk_chunked(sndr, pol, shape, f)~~The transform_sender overload that accepts senders with tag bulk_chunked_t returns a sender such that if it is connected to a receiver rcvr and the resulting operation state is started, then:

(10.1) — If ~~sndr~~ child completes with values vals, let args be a pack of lvalue subexpressions designating vals, then b.schedule_bulk_chunked(~~shape~~ p ? shape : 1, r, s) is called, where:

(10.1.1) — ~~r is a bulk chunked proxy for rcvr with callable f and arguments args; and~~

(10.1.1) — r is a proxy for rcvr with base system_context_replaceability::bulk_item_receiver_proxy such that r.execute(i, j) for indices i and j has effects equivalent to f(i, j, args...) if p is true and f(0, shape, args...) otherwise; and

(10.1.2) — s is a preallocated backend storage for r.

(10.2) — All other completion operations are forwarded unchanged.

~~[*Note:* Customizing the behavior of bulk_chunked affects the default implementation of bulk — *end note*].~~

11 ~~parallel_scheduler provides a customized implementation of bulk_unchunked algorithm (33.9.12.11 [exec.bulk]). If a receiver rcvr is connected to the sender returned by bulk_unchunked(sndr, pol, shape, f)~~The transform_sender overload that accepts senders with tag bulk_unchunked_t returns a sender such that if it is connected to a receiver rcvr and the resulting operation state is started, then:

(11.1) — If ~~sndr~~ child completes with values vals, let args be a pack of lvalue subexpressions designating vals, then b.schedule_bulk_unchunked(~~shape~~ p ? shape : 1, r, s) is called, where:

(11.1.1) — ~~r is a bulk unchunked proxy for rcvr with callable f and arguments args; and~~

(11.1.1) — `r` is a proxy for `rcvr` with base `system_context_replaceability::bulk_item_receiver_proxy` such that `r.execute(i, i + 1)` for index `i` has effects equivalent to `f(i, args...)` if `p` is true and for `(decltype(shape) i = 0; i < shape; i++) { f(i, args...); }` otherwise; and

(11.1.2) — `s` is a preallocated backend storage for `r`.

(11.2) — All other completion operations are forwarded unchanged.

[Editor's note: In section *query_parallel_scheduler_backend* [exec.sysctxrepl.query], apply the following changes:]

```
namespace std::execution::system_context_replaceability {
  struct receiver_proxy {
    virtual ~receiver_proxy() = default;

  protected:
    virtual bool query-env(unspecified) noexcept = 0;   // exposition only

  public:
    virtual void set_value() noexcept = 0;
    virtual void set_error(exception_ptr) noexcept = 0;
    virtual void set_stopped() noexcept = 0;

    template<class P, class-type Query>
      optional<P> try_query(Query q) const noexcept;
  };

  struct bulk_item_receiver_proxy : receiver_proxy {
    virtual void execute(size_t, size_t) noexcept = 0;
  };
}
```

4 `receiver_proxy` represents a receiver that will be notified by the implementations of `parallel_scheduler_backend` to trigger the completion operations. `bulk_item_receiver_proxy` is derived from `receiver_proxy` and is used for `bulk_chunked` and `bulk_unchunked` customizations that will also receive notifications from implementations of `parallel_scheduler_backend` corresponding to different iterations.

```
template <class P, class-type Query>
optional<P> try_query(Query q) const noexcept;
```

5 *Mandates*: `P` is a cv-unqualified non-array object type.

6 *Returns*: Let `env` be the environment of the receiver represented by `*this`. If:

(6.1) — `Query` is not a member of an implementation-defined set of supported queries; or

(6.2) — `P` is not a member of an implementation-defined set of supported result types for `Query`; or

(6.3) — the expression `q(env)` is not well-formed~~or does not have type *cv* `P`,~~

then returns `nullopt`. Otherwise, if `q(env)` has type *cv* `P`, then returns `q(env)`. Otherwise, returns an implementation-defined value of type `optional<P>`.

7 *Remarks*: `get_stop_token_t` is in the implementation-defined set of supported queries, and `inplace_stop_token` is a member of the implementation-defined set of supported result types for `get_stop_token_t`.

8 *Recommended practice*: If `P` is `inplace_stop_token` and `T` is a type other than `inplace_stop_token` that models `stoppable_token`, `try_query` should return an object of type `inplace_stop_token` such that until one of `set_value`, `set_error` or `set_stopped` is called on `*this`, all calls to `try_query` return equivalent `inplace_stop_token` objects.

§ 5 Polls

§ 5.1 SG1 + LEWG, Kona 2025

1. Change namespace “system_context_replacability” to “replacement” as proposed in P3804R0 Iterating on parallel_scheduler.

```
| SF | F | N | A | SA |
| 2  | 1 | 6 | 5 | 1  |
```

No consensus for a change (leave the namespace name)

2. We would like to follow the option “request_stop” virtual function as discussed in the alternative in P3804R0 Iterating on parallel_scheduler (requiring additional paper).

```
| SF | F | N | A | SA |
| 0  | 2 | 5 | 1 | 1  |
```

No consensus for a change

3. Apply the changes (not changing the namespace name) and apply the current suggested fix for try_query (in 2.3) as proposed in the paper P3804R0 and forward the “RO 4-395 33.15 [exec.par.schedule] Improve parallel_scheduler P3804” NB comment to LWG.

```
| SF | F | N | A | SA |
| 5  | 4 | 3 | 0 | 0  |
```

Weak consensus in favor (due to low voting rate)

§ 6 Acknowledgments

Thanks to Tim Song and Tomasz Kamiński for working extra time to ensure that [P2079R10] is specified with high standards, for their love and care for the standard.

Thanks to Lewis Baker for constantly pushing to get better and better solutions to the problems at hand.

§ 7 References

[P2079R10] Lucian Radu Teodorescu, Ruslan Arutyunyan, Lee Howes, Michael Voss. 2025-06-25. Parallel Scheduler.
https://wg21.link/p2079r10

[P3826R1] Eric Niebler. Fix or Remove Sender Algorithm Customization.
https://isocpp.org/files/papers/P3826R1.html